UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Parking Drill Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Parking Drill

Parking Drill is a standalone editable Unity game template. Use the steering and gas controls to drive into the marked parking bay. Brake, reverse and hold the required parking position while avoiding cars and kerbs. Three strikes end a lesson; individual lots introduce their own obstacles and rules.

Includes 25 curated levels, source-backed generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Parking Drill > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/ParkAcademy/Scenes/ParkAcademy_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/ParkAcademy`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/parkacademy contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/PaData.cs`
- `Scripts/PaGen.cs`
- `Scripts/PaLevels.cs`
- `Scripts/PaMesh.cs`
- `Scripts/PaPlanner.cs`
- `Scripts/ParkAcademyGame.cs`
- `Scripts/ParkAcademyGame.UI.cs`
- `Scripts/ParkAcademyGame.View.cs`
- `Scripts/PaSim.cs`

Main flow: ParkAcademyGame.cs. UI and popup consumers: ParkAcademyGame.UI.cs. Model/generator/loading: PaData.cs, PaGen.cs, PaLevels.cs, PaSim.cs. Authoring: Editor/ParkAcademyLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Use the steering and gas controls to drive into the marked parking bay. Brake, reverse and hold the required parking position while avoiding cars and kerbs. Three strikes end a lesson; individual lots introduce their own obstacles and rules.

The initial bank has 25 curated levels. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. Exact helper names/unlocks/charges: Scripts/ParkAcademyGame.cs; help and effect descriptions: Scripts/ParkAcademyGame.cs.

Helper	Unlock	Free charges	Action and effect
Guide Line	3	3	Golden dots on the ground show a route that parks the car. Lasts this attempt.
Bumper	6	3	A bubble shield: your next 2 bumps cost no strike.
Slow Tide	10	2	Vans, gulls and lights move at half speed for 15 seconds.
Autopilot	15	2	Coach Pelly drives and parks the car for you (1 star).

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. A successful use requires the stated preconditions from this game: show both activation and its actual effect. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Parking Drill > Level Editor**. Exact implementation: `Editor/ParkAcademyLevelEditor.cs`.

Duplicate an early entry in this game-specific visual editor, change its visible name and one rule, save, validate, then play it. Use the exact source-control table below for this game.

1. Back up Resources/parkacademy/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`ParkAcademyLevelEditor.cs:44`	`[MenuItem("Szekipapa77/Parking Drill/Level Editor")]`
`ParkAcademyLevelEditor.cs:99`	`if (GUILayout.Button("Save", EditorStyles.toolbarButton, GUILayout.Width(60))) Save();`
`ParkAcademyLevelEditor.cs:100`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!_dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel"))) Load();`
`ParkAcademyLevelEditor.cs:102`	`if (GUILayout.Button("Validate", EditorStyles.toolbarButton, GUILayout.Width(70))) ValidateSelected();`
`ParkAcademyLevelEditor.cs:103`	`if (GUILayout.Button("Validate All", EditorStyles.toolbarButton, GUILayout.Width(90))) ValidateAllInWindow();`
`ParkAcademyLevelEditor.cs:104`	`if (GUILayout.Button("Play this level", EditorStyles.toolbarButton, GUILayout.Width(100))) PlayLevel();`

ParkAcademyLevelEditor.cs:115	EditorGUILayout.HelpBox("Drag items to move. Wheel over an item rotates it (Shift = fine). Right-click deletes. " +
ParkAcademyLevelEditor.cs:129	if (GUILayout.Button("Add")) { Record("Add level"); var L = PaGen.Generate(_st.levels.Count + 1, UnityEngine.Random.Range(1, 99999), 0.3f, false, (int)Tpl.OpenLot); PaGen.Prove(L); _st.levels.Insert(Mathf.Min(_st.selected + 1, _st.levels.Count), L); _st.selected = Mathf.Min(_st.selected + 1, _st.levels.Count - 1); }
ParkAcademyLevelEditor.cs:130	if (GUILayout.Button("Dup") && Cur != null) { Record("Duplicate level"); var c = Cur.Clone(); c.name += " copy"; _st.levels.Insert(_st.selected + 1, c); _st.selected++; }
ParkAcademyLevelEditor.cs:131	if (GUILayout.Button("Del") && Cur != null && EditorUtility.DisplayDialog("Delete", "Delete level " + (_st.selected + 1) + "?", "Delete", "Cancel")) { Record("Delete level"); _st.levels.RemoveAt(_st.selected); _st.selected = Mathf.Clamp(_st.selected - 1, 0, _st.levels.Count - 1); }
ParkAcademyLevelEditor.cs:135	if (GUILayout.Button("Up") && _st.selected > 0) { Record("Move up"); (_st.levels[_st.selected - 1], _st.levels[_st.selected]) = (_st.levels[_st.selected], _st.levels[_st.selected - 1]); _st.selected--; }
ParkAcademyLevelEditor.cs:136	if (GUILayout.Button("Down") && _st.selected < _st.levels.Count - 1) { Record("Move down"); (_st.levels[_st.selected + 1], _st.levels[_st.selected]) = (_st.levels[_st.selected], _st.levels[_st.selected + 1]); _st.selected++; }
ParkAcademyLevelEditor.cs:172	EditorGUILayout.LabelField("Auto Generate", EditorStyles.boldLabel);
ParkAcademyLevelEditor.cs:173	_genCount = EditorGUILayout.IntField("Count", _genCount);
ParkAcademyLevelEditor.cs:174	_genStart = EditorGUILayout.IntField("Start level #", _genStart);

ParkAcademyLevelEditor.cs:175	`_genSeed = EditorGUILayout.IntField("Seed", _genSeed);`
ParkAcademyLevelEditor.cs:176	`_genD0 = EditorGUILayout.Slider("Start difficulty", _genD0, 0, 1);`
ParkAcademyLevelEditor.cs:177	`_genD1 = EditorGUILayout.Slider("End difficulty", _genD1, 0, 1);`
ParkAcademyLevelEditor.cs:178	`_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 2, 8);`
ParkAcademyLevelEditor.cs:179	`_genTpl = EditorGUILayout.Popup("Template", _genTpl + 1, TplNames) - 1;`
ParkAcademyLevelEditor.cs:180	`_genReplace = EditorGUILayout.Toggle("Replace range", _genReplace);`
ParkAcademyLevelEditor.cs:191	`if (GUILayout.Button("Generate (validated)")) Generate();`
ParkAcademyLevelEditor.cs:468	`var name = EditorGUILayout.TextField("Name", L.name);`
ParkAcademyLevelEditor.cs:469	`var hint = EditorGUILayout.TextField("Hint", L.hint);`
ParkAcademyLevelEditor.cs:470	`int theme = EditorGUILayout.IntSlider("Theme", L.theme, 0, 2);`
ParkAcademyLevelEditor.cs:471	`bool breather = EditorGUILayout.Toggle("Breather", L.breather);`
ParkAcademyLevelEditor.cs:472	`int strikes = EditorGUILayout.IntSlider("Strike limit", L.strikes, 1, 5);`
ParkAcademyLevelEditor.cs:473	`float w = EditorGUILayout.Slider("Lot width", L.w, 10, 44);`
ParkAcademyLevelEditor.cs:474	`float h = EditorGUILayout.Slider("Lot height", L.h, 10, 44);`
ParkAcademyLevelEditor.cs:475	`EditorGUILayout.LabelField("Bay", EditorStyles.boldLabel);`
ParkAcademyLevelEditor.cs:476	`float bl = EditorGUILayout.Slider("Bay length", L.bay.l, 4.4f, 8f);`

`ParkAcademyLevelEditor.cs:477`	`float bw = EditorGUILayout.Slider("Bay width", L.bay.w, 2.0f, 4f);`
`ParkAcademyLevelEditor.cs:478`	`bool facing = EditorGUILayout.Toggle("Back-in (arrow)", L.bay.facing);`
`ParkAcademyLevelEditor.cs:486`	`EditorGUILayout.LabelField("Add", EditorStyles.boldLabel);`
`ParkAcademyLevelEditor.cs:489`	`if (GUILayout.Button("Car")) AddBox(L, BoxKind.ParkedCar, 4.1f, 1.85f);`
`ParkAcademyLevelEditor.cs:490`	`if (GUILayout.Button("Cone")) AddBox(L, BoxKind.Cone, 0.55f, 0.55f);`
`ParkAcademyLevelEditor.cs:491`	`if (GUILayout.Button("Planter")) AddBox(L, BoxKind.Planter, 3f, 1.2f);`
`ParkAcademyLevelEditor.cs:495`	`if (GUILayout.Button("Kerb")) AddBox(L, BoxKind.Kerb, 6f, 1.4f);`
`ParkAcademyLevelEditor.cs:496`	`if (GUILayout.Button("Wall")) AddBox(L, BoxKind.Wall, 6f, 0.9f);`
`ParkAcademyLevelEditor.cs:497`	`if (GUILayout.Button("Bin")) AddBox(L, BoxKind.Bin, 0.9f, 0.9f);`
`ParkAcademyLevelEditor.cs:501`	`if (GUILayout.Button("Booth")) AddStop(L, StopKind.Booth);`
`ParkAcademyLevelEditor.cs:502`	`if (GUILayout.Button("Wash")) AddStop(L, StopKind.Wash);`
`ParkAcademyLevelEditor.cs:503`	`if (GUILayout.Button("Crossing")) { Record("Add crossing"); L.crossings = Add(L.crossings, new PaCross { x = 0, y = 0, a = 0, l = L.w, w = 3.2f }); }`
`ParkAcademyLevelEditor.cs:507`	`if (GUILayout.Button("Van lane")) { Record("Add van"); L.movers = Add(L.movers, new PaMover { xs = new[] { -L.w / 2 - 2, L.w / 2 + 2 }, ys = new[] { 0f, 0f }, speed = 4, phase = 0 }); }`
`ParkAcademyLevelEditor.cs:508`	`if (GUILayout.Button("Puddle")) { Record("Add puddle"); L.puddles = Add(L.puddles, new PaPuddle { x = 0, y = 0, r = 1.8f }); }`
`ParkAcademyLevelEditor.cs:509`	`if (GUILayout.Button("Arrow")) { Record("Add arrow"); L.arrows = Add(L.arrows, new PaArrow { x = 0, y = 0, a = PaMath.HalfPI }); }`

ParkAcademyLevelEditor.cs:514	`EditorGUILayout.LabelField("Bot", EditorStyles.boldLabel);`
ParkAcademyLevelEditor.cs:515	`EditorGUILayout.LabelField($"score {L.score} plan {L.planTicks / 60f:0.0}s {(string.IsNullOrEmpty(L.plan) ? "UNPROVEN" : "proven")}");`
ParkAcademyLevelEditor.cs:516	`if (GUILayout.Button("Re-roll layout (same template)"))`
ParkAcademyLevelEditor.cs:538	`if (_sel == Sel.None) { EditorGUILayout.HelpBox("Click an item on the lot to edit it.", MessageType.None); return; }`
ParkAcademyLevelEditor.cs:539	`EditorGUILayout.LabelField("Selected: " + _sel + (_selIdx >= 0 ? " " + (_selIdx + 1) : ""), EditorStyles.boldLabel);`
ParkAcademyLevelEditor.cs:546	`int kind = EditorGUILayout.Popup("Kind", b.kind, BoxNames);`
ParkAcademyLevelEditor.cs:547	`float l = EditorGUILayout.Slider("Length", b.l, 0.3f, 30f), w = EditorGUILayout.Slider("Width", b.w, 0.3f, 12f);`
ParkAcademyLevelEditor.cs:548	`float a = EditorGUILayout.Slider("Angle", b.a * Mathf.Rad2Deg, -180, 180) * Mathf.Deg2Rad;`
ParkAcademyLevelEditor.cs:549	`int tint = EditorGUILayout.IntSlider("Tint", b.tint, 0, 5);`
ParkAcademyLevelEditor.cs:557	`int kind = EditorGUILayout.Popup("Kind", s.kind, new[] { "Booth", "Wash" });`
ParkAcademyLevelEditor.cs:558	`float l = EditorGUILayout.Slider("Box length", s.l, 4.4f, 8f), w = EditorGUILayout.Slider("Box width", s.w, 2f, 4f);`
ParkAcademyLevelEditor.cs:559	`float gl = EditorGUILayout.Slider("Gate length", s.gate.l, 1f, 8f);`
ParkAcademyLevelEditor.cs:560	`float ga = EditorGUILayout.Slider("Gate angle", s.gate.a * Mathf.Rad2Deg, -180, 180) * Mathf.Deg2Rad;`
ParkAcademyLevelEditor.cs:567	`float l = EditorGUILayout.Slider("Length", c.l, 4f, 44f), w = EditorGUILayout.Slider("Width", c.w, 2f, 5f);`

`ParkAcademyLevelEditor.cs:568`	`float g = EditorGUILayout.Slider("Green s", c.green, 1f, 10f), am = EditorGUILayout.Slider("Amber s", c.amber, 0.3f, 3f), r = EditorGUILayout.Slider("Red s", c.red, 1f, 10f);`
`ParkAcademyLevelEditor.cs:569`	`float off = EditorGUILayout.Slider("Offset s", c.offset, 0f, 20f);`
`ParkAcademyLevelEditor.cs:576`	`float sp = EditorGUILayout.Slider("Speed m/s", m.speed, 1f, 9f), ph = EditorGUILayout.Slider("Phase m", m.phase, 0f, 60f);`
`ParkAcademyLevelEditor.cs:577`	`bool loop = EditorGUILayout.Toggle("Loop", m.loop);`
`ParkAcademyLevelEditor.cs:578`	`int tint = EditorGUILayout.IntSlider("Tint", m.tint, 0, 5);`
`ParkAcademyLevelEditor.cs:579`	`bool addPt = GUILayout.Button("Add point");`
`ParkAcademyLevelEditor.cs:591`	`float r = EditorGUILayout.Slider("Radius", p.r, 0.6f, 4f);`
`ParkAcademyLevelEditor.cs:597`	`float a = EditorGUILayout.Slider("Heading", L.start.a * Mathf.Rad2Deg, -180, 180) * Mathf.Deg2Rad;`
`ParkAcademyLevelEditor.cs:603`	`float a = EditorGUILayout.Slider("Angle", L.bay.a * Mathf.Rad2Deg, -180, 180) * Mathf.Deg2Rad;`

CHAPTER 07

Generator, insertion and replacement

Use this editor generation panel, choose a reproducible seed and small count, review start/index and replace behavior, then generate, save and reopen. The source excerpt below identifies exact append/replace semantics.

Source	Exact current code
`ParkAcademyLevelEditor.cs:40`	`float _genD0 = 0.4f, _genD1 = 0.9f; bool _genReplace;`
`ParkAcademyLevelEditor.cs:77`	`if (f?.levels != null) _st.levels.AddRange(f.levels);`
`ParkAcademyLevelEditor.cs:180`	`_genReplace = EditorGUILayout.Toggle("Replace range", _genReplace);`
`ParkAcademyLevelEditor.cs:210`	`if (_genReplace) { int rem = Mathf.Min(_genCount, _st.levels.Count - at); if (rem > 0) _st.levels.RemoveRange(at, rem); }`
`ParkAcademyLevelEditor.cs:211`	`_st.levels.InsertRange(at, made);`

Worked extension: back up the bank, generate a small set at moderate difficulty using the exact per-game controls, validate them, then Save. Reopen the JSON and confirm the count changed. Do not assume every editor has the same insertion behavior: the actual implementation below distinguishes insert, append and replace. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/ParkAcademy/Resources/parkacademy/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
  "Dynamic": false,
  "name": "First Drive",
  "hint": "Drive into the glowing bay and stop inside the lines.",
  "theme": 0,
  "tpl": 0,
  "seed": 11,
  "difficulty": 0,
  "breather": false,
  "strikes": 3,
  "w": 20,
  "h": 26,
  "start": {
    "x": -1.4,
    "y": -9.5,
    "a": 1.5707964
  },
  "bay": {
    "x": 2.2,
    "y": 8.6,
    "a": 1.5707964,
    "l": 5.9,
    "w": 3.2,
    "facing": false
  },
  "boxes": [
    {
      "kind": 1,
      "x": -1.8999999,
      "y": 8.6,
      "a": 1.5707964,
      "l": 4.1,
      "w": 1.85,
      "tint": 2
    },
    {
      "kind": 1,
      "x": 6.3,
      "y": 8.6,
      "a": -1.5707964,
      "l": 4.1,
      "w": 1.85,
      "tint": 4
    }
  ],
  "stops": [],
  "movers": [],
  "crossings": [],
  "puddles": [],
  "arrows": [
    {
      "x": -5.2301426,
      "y": -2.7233062,
      "a": 1.7770369
    },
    {
      "x": -5.8854437,
      "y": 0.4088812,
      "a": 1.7770369
    }
  ],
  "plan": "6:70,7:10,8:30,6:10,7:20,2:10,6:10,2:10,8:10,2:10,8:30,6:10,7:40,1:10,0:10,6:10,0:10,10:67",
  "planTicks": 377,
  "score": 13
}
```

Root keys: levels. Inspect PaData.cs, PaGen.cs, PaLevels.cs, PaSim.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. The game-specific board, wave, card, opponent or question records are gameplay data; UI translation is separate.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/ParkAcademyGame.UI.cs`. Find `Ink`, currently `#24375A`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`ParkAcademyGame.UI.cs:74 / void BuildHud()`	`var set = HzIcons.IconButton(_hud, HzIcons.Settings, Ink, () => { if (_phase != Phase.Ended) ShowSettingsPopup(); }, bs);`
`ParkAcademyGame.UI.cs:101 / void BuildHud()`	`_goalText = HzUI.Label("Goal", _strikePanel, "", 36, Ink, TextAnchor.MiddleLeft, true);`
`ParkAcademyGame.UI.cs:140 / void BuildHud()`	`_tipText = HzUI.Label("T", _tipPanel, "", portrait ? 40 : 38, Ink, TextAnchor.MiddleCenter, true);`
`ParkAcademyGame.UI.cs:482 / HzPopup AcademyPopup(string title, string message, Color accent, Sprite art, params (string label, Color color, Action action)[] buttons)`	`p.Message = HzUI.Label("Msg", card.transform, message, 40, Ink, TextAnchor.MiddleCenter, false);`
`ParkAcademyGame.UI.cs:611 / void ShowSettingsPopup()`	`var p = AcademyPopup("Settings", "", Ink, null,`

Source	Exact current code
`ParkAcademyGame.cs:28 / class/static declarations`	`static readonly string[] BoostColor = { "#FFB938", "#2EC4B6", "#5B9DF0", "#FF7A6B" };`
`ParkAcademyGame.cs:236 / void OnSimEvents(byte code)`	`if (_s.status == 0) Toast(StrikeText(), HzUI.Hex("#FFD1C9"), 1.1f, 0.7f);`
`ParkAcademyGame.cs:319 / IEnumerator FailFlow()`	`buttons.Add(("Try Again", HzUI.Hex("#2EC4B6"), () => LoadLevel(Level, true)));`
`ParkAcademyGame.cs:321 / IEnumerator FailFlow()`	`buttons.Add(("Keep Driving (Ad)", HzUI.Hex("#FFB938"), () => HzPlatform.Rewarded(GameId, "rescue", ok => { if (ok) Rescue(); else LoadLevel(Level, true); })));`

ParkAcademyGame.cs:323 / IEnumerator FailFlow()	buttons.Add(("Skip Lesson (Ad)", HzUI.Hex("#9D8CFF"), () => HzPlatform.Rewarded(GameId, "skip", ok => { if (ok) { Level = Level + 1; LoadLevel(Level); } else LoadLevel(Level, true); })));
ParkAcademyGame.cs:368 / void OnBooster(int b)	("Watch Ad", HzUI.Hex("#2EC4B6"), () => HzPlatform.Rewarded(GameId, "booster_" + b, ok => { if (ok) { SetCharges(b, Charges(b) + 1); HzAudio.Play(Sfx.Coin, 0.7f); RefreshBoosters(); } })),
ParkAcademyGame.cs:369 / void OnBooster(int b)	("No thanks", HzUI.Hex("#B8A99A"), null));
ParkAcademyGame.cs:393 / void StartAutopilot()	Toast("Coach Pelly is driving!", HzUI.Hex("#FFE3D9"), 1.6f, 0.62f);
ParkAcademyGame.UI.cs:12 / class/static declarations	static readonly Color Ink = HzUI.Hex("#24375A");
ParkAcademyGame.UI.cs:13 / class/static declarations	static readonly Color Coral = HzUI.Hex("#FF7A6B");
ParkAcademyGame.UI.cs:14 / class/static declarations	static readonly Color Teal = HzUI.Hex("#2EC4B6");
ParkAcademyGame.UI.cs:15 / class/static declarations	static readonly Color Cream = HzUI.Hex("#FFF6E5");
ParkAcademyGame.UI.cs:68 / void BuildHud()	_levelSub = HzUI.Label("Name", lp.transform, _def.name, 30, HzUI.Hex("#E3FFFB"), TextAnchor.MiddleCenter, false);
ParkAcademyGame.UI.cs:76 / void BuildHud()	var cam = HzIcons.IconButton(_hud, _camMode == 0 ? HzIcons.Eye : HzIcons.Grid, HzUI.Hex("#5B9DF0"), ToggleCamera, bs);
ParkAcademyGame.UI.cs:149 / void BuildHud()	_ringFill = HzUI.Img("Fill", _ring, HzUI.RoundRect(64, 14), HzUI.Hex("#FFC94A")); _ringFill.raycastTarget = false; _ringFill.rectTransform.Stretch();
ParkAcademyGame.UI.cs:171 / void BuildPads(bool portrait)	_padLeft = MakePad("SteerL", HzIcons.ArrowBack, HzUI.Hex("#5B9DF0"), new Vector2(0, 0), new Vector2(pad, pad), new Vector2(Side + pad / 2, py), "");

`ParkAcademyGame.UI.cs:172 / void BuildPads(bool portrait)`	`_padRight = MakePad("SteerR", HzIcons.ArrowForward, HzUI.Hex("#5B9DF0"), new Vector2(0, 0), new Vector2(pad, pad), new Vector2(Side + pad * 1.5f + 30, py), "");`
`ParkAcademyGame.UI.cs:175 / void BuildPads(bool portrait)`	`_padGas = MakePad("Gas", HzIcons.Play, HzUI.Hex("#43CF8A"), new Vector2(1, 0), gasSize, new Vector2(-Side - gasSize.x / 2, SafeBottom + gasSize.y / 2 + 20), "GAS");`
`ParkAcademyGame.UI.cs:224 / void BuildPads(bool portrait)`	`var bd = HzUI.Panel("Badge", v.Root, HzUI.Hex("#FF4D6D"), 30); bd.raycastTarget = false;`
`ParkAcademyGame.UI.cs:229 / void BuildPads(bool portrait)`	`var on = HzUI.Panel("On", v.Root, HzUI.Hex("#43CF8A"), 20); on.raycastTarget = false;`
`ParkAcademyGame.UI.cs:247 / void RefreshBoosters()`	`v.Inner.color = un ? HzUI.Hex(BoostColor[b]) : HzUI.Hex("#B9C6D3");`
`ParkAcademyGame.UI.cs:254 / void RefreshBoosters()`	`v.Badge.GetComponent<Image>().color = c > 0 ? HzUI.Hex("#FF4D6D") : HzUI.Hex("#43CF8A");`
`ParkAcademyGame.UI.cs:256 / void RefreshBoosters()`	`v.Ring.color = act || _guide == b ? HzUI.Hex("#FFE066") : new Color(1, 1, 1, 0);`
`ParkAcademyGame.UI.cs:280 / void UpdateHud()`	`_ringFill.color = Color.Lerp(HzUI.Hex("#FFC94A"), HzUI.Hex("#43CF8A"), f);`
`ParkAcademyGame.UI.cs:320 / void LevelBanner()`	`var n = HzUI.Label("N", bg.transform, sub, 44, HzUI.Hex("#BDF5EC")); n.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(800, 70), new Vector2(0, 18), new Vector2(0.5f, 0));`
`ParkAcademyGame.UI.cs:400 / void CompleteGuide(int b, bool toast = true)`	`if (toast) Toast("Nice! " + BoostName[b] + " is on", HzUI.Hex("#E3FFF6"), 1.3f, 0.62f);`
`ParkAcademyGame.UI.cs:448 / HzPopup AcademyPopup(string title, string message, Color accent, Sprite art, params (string label, Color color, Action action)[] buttons)`	`var card = HzUI.Panel("Card", dim.transform, HzUI.Hex("#FFFBF3"), 56);`
`ParkAcademyGame.UI.cs:451 / HzPopup AcademyPopup(string title, string message, Color accent, Sprite art, params (string label, Color color, Action action)[] buttons)`	`var inner = HzUI.Panel("Inner", card.transform, HzUI.Hex("#FFF1E0"), 44); inner.raycastTarget = false;`

`ParkAcademyGame.UI.cs:587 / void WinPopup(int lv, int stars, string msg, Action next)`	`("Next Lesson", HzUI.Hex("#43CF8A"), next));`
`ParkAcademyGame.UI.cs:594 / void WinPopup(int lv, int stars, string msg, Action next)`	`var s = HzIcons.Icon(row, HzIcons.Star, i == 1 ? 104 : 86, got ? HzUI.Hex("#FFC94A") : new Color(0.85f, 0.85f, 0.9f));`
`ParkAcademyGame.UI.cs:612 / void ShowSettingsPopup()`	`(SoundLabel(), HzUI.Hex("#5B9DF0"), null),`
`ParkAcademyGame.UI.cs:613 / void ShowSettingsPopup()`	`(MusicLabel(), HzUI.Hex("#9D8CFF"), null),`
`ParkAcademyGame.UI.cs:614 / void ShowSettingsPopup()`	`("How to Play", HzUI.Hex("#FFB938"), ShowHowTo),`
`ParkAcademyGame.UI.cs:629 / void ShowHowTo()`	`var p = AcademyPopup("How to Play", text, HzUI.Hex("#FFB938"), Art("coach_pelly"), ("Let's drive!", Teal, null));`
`ParkAcademyGame.View.cs:31 / class/static declarations`	`static readonly string[] CarTints = { "#FF8A7A", "#5B9DF0", "#FFC94A", "#9D8CFF", "#F7F2E8", "#FF9FC8" };`
`ParkAcademyGame.View.cs:32 / class/static declarations`	`static readonly string[] Paving = { "#E8B49C", "#A9C0D8", "#E4C493" };`
`ParkAcademyGame.View.cs:33 / class/static declarations`	`static readonly string[] PavingDark = { "#D99A80", "#8FA9C4", "#D3AE78" };`
`ParkAcademyGame.View.cs:34 / class/static declarations`	`static readonly string[] Ground = { "#EDD49B", "#E2CFA5", "#EFCF94" };`
`ParkAcademyGame.View.cs:41 / void SetupLook()`	`_cam.backgroundColor = HzUI.Hex("#9ED8F5");`
`ParkAcademyGame.View.cs:51 / void SetupLook()`	`_sun.color = HzUI.Hex("#FFF0D8"); _sun.intensity = 1.1f;`
`ParkAcademyGame.View.cs:59 / void SetupLook()`	`fill.transform.rotation = Quaternion.Euler(28, 150, 0); fill.color = HzUI.Hex("#BDE6FF"); fill.intensity = 0.35f;`
`ParkAcademyGame.View.cs:62 / void SetupLook()`	`RenderSettings.ambientSkyColor = HzUI.Hex("#CFEFFF");`
`ParkAcademyGame.View.cs:63 / void SetupLook()`	`RenderSettings.ambientEquatorColor = HzUI.Hex("#FFE2C8");`

ParkAcademyGame.View.cs:64 / void SetupLook()	RenderSettings.ambientGroundColor = HzUI.Hex("#B89C7A");
ParkAcademyGame.View.cs:68 / void SetupLook()	RenderSettings.fogColor = HzUI.Hex("#BFE6F7");
ParkAcademyGame.View.cs:77 / void SetupLook()	if (prof.TryGet<Vignette>(out var vg)) { vg.intensity.Override(0.16f); vg.color.Override(HzUI.Hex("#3A5068")); }
ParkAcademyGame.View.cs:186 / void BuildWorld()	MeshObj("Sea", _world, PaMesh.Quad(400, 160), Lit("#3CC7C0", 0.85f), W(0, hh + 28 + 80, -0.12f), Quaternion.identity, false, true);
ParkAcademyGame.View.cs:187 / void BuildWorld()	MeshObj("Foam", _world, PaMesh.Quad(400, 1.2f), Unlit("#E9FBF8"), W(0, hh + 28, -0.05f), Quaternion.identity, false, false);
ParkAcademyGame.View.cs:188 / void BuildWorld()	MeshObj("WetSand", _world, PaMesh.Quad(400, 5f), Lit("#E6CF9C", 0.5f), W(0, hh + 25.5f, -0.04f), Quaternion.identity, false, true);
ParkAcademyGame.View.cs:197 / void BuildWorld()	var kerb = Lit("#FFF6E5", 0.35f); var stripe = Lit("#FF7A6B", 0.4f);
ParkAcademyGame.View.cs:206 / void BuildWorld()	MeshObj("Arrow", _world, PaMesh.Arrow(2.2f, 1.3f), Unlit("#FFFFFF", 0.95f), W(a.x, a.y, 0.012f), Yaw(a.a), false, false);
ParkAcademyGame.View.cs:211 / void BuildWorld()	MeshObj("Puddle", _world, PaMesh.Disc(p.r), Glass("#7FC8E8", 0.55f), W(p.x, p.y, 0.015f), Quaternion.identity, false, true);
ParkAcademyGame.View.cs:212 / void BuildWorld()	MeshObj("PuddleRim", _world, PaMesh.Ring(p.r * 0.92f, p.r), Fade("#FFFFFF", 0.35f), W(p.x, p.y, 0.02f), Quaternion.identity, false, false);
ParkAcademyGame.View.cs:239 / void BuildBay()	_bayFill = MeshObj("Fill", _bayRoot, PaMesh.Quad(b.w, b.l), Fade("#2EC4B6", 0.22f), Vector3.zero, Quaternion.identity, false, false).GetComponent<Renderer>();
ParkAcademyGame.View.cs:240 / void BuildBay()	_bayLines = RectLines(_bayRoot, b.w, b.l, 0.14f, Emit("#B8FFF2", 1.6f));
ParkAcademyGame.View.cs:242 / void BuildBay()	if (b.facing) MeshObj("BackArrow", _bayRoot, PaMesh.Arrow(b.l * 0.55f, b.w * 0.55f), Unlit("#FFFFFF", 1f), Vector3.up * 0.01f, Quaternion.identity, false, false);
ParkAcademyGame.View.cs:246 / void BuildBay()	var m = Unlit("#FFFFFF", 1f);

`ParkAcademyGame.View.cs:271 / void BuildStop(int i)`	`MeshObj("Fill", root, PaMesh.Quad(s.w, s.l), Fade(wash ? "#7FC8FF" : "#FFC94A", 0.25f), Vector3.zero, Quaternion.identity, false, false);`
`ParkAcademyGame.View.cs:272 / void BuildStop(int i)`	`_stopLines.Add(RectLines(root, s.w, s.l, 0.14f, Emit(wash ? "#9FDBFF" : "#FFE08A", 1.4f)));`
`ParkAcademyGame.View.cs:274 / void BuildStop(int i)`	`for (int k = -2; k <= 2; k++) MeshObj("Hatch", root, PaMesh.Quad(s.w * 0.8f, 0.12f), Fade("#FFFFFF", 0.55f), new Vector3(0, 0.008f, k * s.l * 0.16f), Quaternion.Euler(0, 25, 0), false, false);`
`ParkAcademyGame.View.cs:278 / void BuildStop(int i)`	`var arch = Lit("#5B9DF0", 0.5f);`
`ParkAcademyGame.View.cs:281 / void BuildStop(int i)`	`MeshObj("ArchTop", root, PaMesh.RoundBox(new Vector3(s.w + 1.3f, 0.45f, 1.4f), 0.15f, 2), Lit("#FFFFFF", 0.5f), new Vector3(0, 3.05f, 0));`
`ParkAcademyGame.View.cs:284 / void BuildStop(int i)`	`var br = MeshObj("Brush", root, PaMesh.Cylinder(0.32f, 2.2f, 12), Lit(k < 0 ? "#FF7A6B" : "#FFC94A", 0.2f), new Vector3(k * (s.w / 2 + 0.05f), 1.3f, 0.2f));`
`ParkAcademyGame.View.cs:292 / void BuildStop(int i)`	`MeshObj("Hut", hut, PaMesh.RoundBox(new Vector3(1.6f, 2.0f, 1.8f), 0.12f, 2), Lit("#FFF6E5", 0.4f), new Vector3(0, 1.0f, 0));`
`ParkAcademyGame.View.cs:293 / void BuildStop(int i)`	`MeshObj("Window", hut, PaMesh.Box(new Vector3(0.05f, 0.7f, 1.2f)), Glass("#2A4D69", 0.85f), new Vector3(-0.81f, 1.3f, 0));`
`ParkAcademyGame.View.cs:294 / void BuildStop(int i)`	`MeshObj("Roof", hut, PaMesh.RoundBox(new Vector3(2.0f, 0.25f, 2.2f), 0.1f, 2), Lit("#FF7A6B", 0.5f), new Vector3(0, 2.1f, 0));`
`ParkAcademyGame.View.cs:301 / void BuildStop(int i)`	`MeshObj("Post", gate, PaMesh.RoundBox(new Vector3(0.45f, 1.2f, 0.45f), 0.08f, 2), Lit("#24375A", 0.4f), new Vector3(-len / 2 - 0.3f, 0.6f, 0));`
`ParkAcademyGame.View.cs:304 / void BuildStop(int i)`	`MeshObj("Seg", arm, PaMesh.RoundBox(new Vector3(len / 6f, 0.2f, 0.2f), 0.06f, 2), Lit(k % 2 == 0 ? "#FF5A4E" : "#FFFFFF", 0.5f), new Vector3(len / 12f + k * len / 6f + 0.1f, 0, 0));`

`ParkAcademyGame.View.cs:317 / void BuildBox(PaBox b)`	`MeshObj("Base", t, PaMesh.RoundBox(new Vector3(0.55f, 0.06f, 0.55f), 0.03f, 1), Lit("#FF6A3D", 0.3f), new Vector3(0, 0.03f, 0));`
`ParkAcademyGame.View.cs:318 / void BuildBox(PaBox b)`	`MeshObj("Cone", t, PaMesh.Cone(0.22f, 0.75f, 14), Lit("#FF7A45", 0.45f), new Vector3(0, 0.43f, 0));`
`ParkAcademyGame.View.cs:319 / void BuildBox(PaBox b)`	`MeshObj("Band", t, PaMesh.Cylinder(0.155f, 0.12f, 14, 0.8f), Lit("#FFFFFF", 0.5f), new Vector3(0, 0.5f, 0), Quaternion.identity, false, true);`
`ParkAcademyGame.View.cs:322 / void BuildBox(PaBox b)`	`MeshObj("Box", t, PaMesh.RoundBox(new Vector3(b.w, 0.55f, b.l), 0.12f, 2), Lit("#E9D2B0", 0.3f), new Vector3(0, 0.275f, 0));`
`ParkAcademyGame.View.cs:327 / void BuildBox(PaBox b)`	`MeshObj("Bush", t, PaMesh.Sphere(0.5f, 12, 8), Lit(i % 2 == 0 ? "#5CC98A" : "#4DB87C", 0.25f), new Vector3(0, 0.75f, z), Quaternion.identity).localScale = new Vector3(b.w * 0.95f, 0.8f, b.l / n * 1.15f);`
`ParkAcademyGame.View.cs:331 / void BuildBox(PaBox b)`	`MeshObj("Bin", t, PaMesh.Cylinder(b.w * 0.45f, 0.95f, 14), Lit("#2EC4B6", 0.5f), new Vector3(0, 0.475f, 0));`
`ParkAcademyGame.View.cs:332 / void BuildBox(PaBox b)`	`MeshObj("Lid", t, PaMesh.Cylinder(b.w * 0.5f, 0.1f, 14), Lit("#24375A", 0.5f), new Vector3(0, 1.0f, 0));`
`ParkAcademyGame.View.cs:335 / void BuildBox(PaBox b)`	`MeshObj("Crate", t, PaMesh.RoundBox(new Vector3(b.w, 0.9f, b.l), 0.06f, 2), Lit("#D9A066", 0.25f), new Vector3(0, 0.45f, 0));`
`ParkAcademyGame.View.cs:338 / void BuildBox(PaBox b)`	`MeshObj("Wall", t, PaMesh.RoundBox(new Vector3(b.w, 0.9f, b.l), 0.12f, 2), Lit("#FFF1DC", 0.35f), new Vector3(0, 0.45f, 0));`
`ParkAcademyGame.View.cs:339 / void BuildBox(PaBox b)`	`MeshObj("Cap", t, PaMesh.RoundBox(new Vector3(b.w + 0.08f, 0.12f, b.l + 0.04f), 0.05f, 1), Lit("#2EC4B6", 0.5f), new Vector3(0, 0.95f, 0));`

`ParkAcademyGame.View.cs:342 / void BuildBox(PaBox b)`	`MeshObj("Island", t, PaMesh.RoundBox(new Vector3(b.w, 0.3f, b.l), 0.12f, 2), Lit("#FFF6E5", 0.35f), new Vector3(0, 0.15f, 0));`
`ParkAcademyGame.View.cs:343 / void BuildBox(PaBox b)`	`MeshObj("Grass", t, PaMesh.RoundBox(new Vector3(Mathf.Max(0.1f, b.w - 0.3f), 0.08f, Mathf.Max(0.1f, b.l - 0.3f)), 0.04f, 1), Lit("#6ED39A", 0.2f), new Vector3(0, 0.32f, 0), Quaternion.identity, false, true);`
`ParkAcademyGame.View.cs:355 / void BuildCrossing(int i)`	`var white = Unlit("#FFFFFF", 0.97f);`
`ParkAcademyGame.View.cs:368 / void BuildCrossing(int i)`	`MeshObj("Stick", pole, PaMesh.Cylinder(0.09f, 2.6f, 10), Lit("#24375A", 0.5f), new Vector3(0, 1.3f, 0));`
`ParkAcademyGame.View.cs:369 / void BuildCrossing(int i)`	`MeshObj("Head", pole, PaMesh.RoundBox(new Vector3(0.5f, 1.2f, 0.45f), 0.1f, 2), Lit("#24375A", 0.6f), new Vector3(0, 2.9f, 0));`
`ParkAcademyGame.View.cs:370 / void BuildCrossing(int i)`	`string[] cols = { "#FF4D4D", "#FFC94A", "#4DE08A" };`
`ParkAcademyGame.View.cs:372 / void BuildCrossing(int i)`	`lamps[side * 3 + k] = MeshObj("Lamp", pole, PaMesh.Sphere(0.15f, 10, 6), Lit("#333C48", 0.6f), new Vector3(0, 3.3f - k * 0.38f, -0.2f), Quaternion.identity, false, false).GetComponent<Renderer>();`
`ParkAcademyGame.View.cs:384 / void BuildCrossing(int i)`	`MeshObj("Body", g, PaMesh.Sphere(0.35f, 12, 8), Lit("#FFFFFF", 0.4f), new Vector3(0, 0.45f, 0)).localScale = new Vector3(0.9f, 0.85f, 1.35f);`
`ParkAcademyGame.View.cs:385 / void BuildCrossing(int i)`	`MeshObj("Wing", g, PaMesh.Sphere(0.25f, 10, 6), Lit("#B8C4D0", 0.4f), new Vector3(0, 0.55f, -0.1f)).localScale = new Vector3(1.6f, 0.45f, 1.4f);`
`ParkAcademyGame.View.cs:386 / void BuildCrossing(int i)`	`MeshObj("Head", g, PaMesh.Sphere(0.2f, 10, 6), Lit("#FFFFFF", 0.4f), new Vector3(0, 0.85f, 0.32f));`
`ParkAcademyGame.View.cs:387 / void BuildCrossing(int i)`	`MeshObj("Beak", g, PaMesh.Cone(0.07f, 0.3f, 8), Lit("#FFB938", 0.4f), new Vector3(0, 0.83f, 0.55f), Quaternion.Euler(90, 0, 0));`

ParkAcademyGame.View.cs:388 / void BuildCrossing(int i)	MeshObj("EyeL", g, PaMesh.Sphere(0.035f, 6, 4), Unlit("#1B2333"), new Vector3(-0.1f, 0.92f, 0.45f), Quaternion.identity, false, false);
ParkAcademyGame.View.cs:389 / void BuildCrossing(int i)	MeshObj("EyeR", g, PaMesh.Sphere(0.035f, 6, 4), Unlit("#1B2333"), new Vector3(0.1f, 0.92f, 0.45f), Quaternion.identity, false, false);
ParkAcademyGame.View.cs:390 / void BuildCrossing(int i)	MeshObj("LegL", g, PaMesh.Cylinder(0.03f, 0.3f, 6), Lit("#FFB938", 0.3f), new Vector3(-0.1f, 0.15f, 0));
ParkAcademyGame.View.cs:391 / void BuildCrossing(int i)	MeshObj("LegR", g, PaMesh.Cylinder(0.03f, 0.3f, 6), Lit("#FFB938", 0.3f), new Vector3(0.1f, 0.15f, 0));
ParkAcademyGame.View.cs:400 / void BuildCrossing(int i)	MeshObj("Stripe", t, PaMesh.RoundBox(new Vector3(1.77f, 0.22f, 3.6f), 0.08f, 2), Lit("#FFFFFF", 0.5f), new Vector3(0, 0.95f, -0.1f), Quaternion.identity, false, true);
ParkAcademyGame.View.cs:401 / void BuildCrossing(int i)	MeshObj("Wind", t, PaMesh.RoundBox(new Vector3(1.6f, 0.55f, 0.2f), 0.1f, 2), Glass("#2A4D69", 0.9f), new Vector3(0, 1.45f, 1.95f), Quaternion.Euler(-12, 0, 0));
ParkAcademyGame.View.cs:402 / void BuildCrossing(int i)	MeshObj("Hatch", t, PaMesh.Box(new Vector3(0.05f, 0.6f, 1.6f)), Glass("#2A4D69", 0.85f), new Vector3(0.88f, 1.35f, -0.4f));
ParkAcademyGame.View.cs:404 / void BuildCrossing(int i)	MeshObj("Cone", t, PaMesh.Cone(0.32f, 0.9f, 12), Lit("#E8B270", 0.3f), new Vector3(0, 2.25f, -0.6f), Quaternion.Euler(180, 0, 0));
ParkAcademyGame.View.cs:405 / void BuildCrossing(int i)	MeshObj("Scoop", t, PaMesh.Sphere(0.4f, 12, 8), Lit("#FF9FC8", 0.4f), new Vector3(0, 2.75f, -0.6f));
ParkAcademyGame.View.cs:407 / void BuildCrossing(int i)	MeshObj("Wheel", t, PaMesh.Cylinder(0.36f, 0.26f, 14), Lit("#2B2F3A", 0.3f), new Vector3((k % 2 == 0 ? -1 : 1) * 0.86f, 0.36f, (k < 2 ? 1 : -1) * 1.3f), Quaternion.Euler(0, 0, 90));
ParkAcademyGame.View.cs:416 / void BuildCarBody(Transform t, string tint, bool player)	MeshObj("Cabin", t, PaMesh.RoundBox(new Vector3(1.6f, 0.62f, 2.1f), 0.28f, 3), Glass("#2A4D69", 0.92f), new Vector3(0, 1.18f, -0.25f));

`ParkAcademyGame.View.cs:420 / void BuildCarBody(Transform t, string tint, bool player)`	`var w = MeshObj("Wheel", t, PaMesh.Cylinder(0.36f, 0.28f, 16), Lit("#2B2F3A", 0.3f), new Vector3((k % 2 == 0 ? -1 : 1) * 0.86f, 0.36f, (k < 2 ? 1 : -1) * 1.3f), Quaternion.Euler(0, 0, 90));`
`ParkAcademyGame.View.cs:421 / void BuildCarBody(Transform t, string tint, bool player)`	`MeshObj("Hub", w, PaMesh.Cylinder(0.18f, 0.3f, 10), Lit("#E8EEF4", 0.7f), Vector3.zero);`
`ParkAcademyGame.View.cs:424 / void BuildCarBody(Transform t, string tint, bool player)`	`var hl = Emit("#FFF6D0", player ? 1.6f : 0.6f);`
`ParkAcademyGame.View.cs:427 / void BuildCarBody(Transform t, string tint, bool player)`	`var tlM = player ? Lit("#B83A3A", 0.6f) : Lit("#D04848", 0.6f);`
`ParkAcademyGame.View.cs:433 / void BuildCarBody(Transform t, string tint, bool player)`	`_brakeOff = tlM; _brakeOn = Emit("#FF3B3B", 2.4f);`
`ParkAcademyGame.View.cs:434 / void BuildCarBody(Transform t, string tint, bool player)`	`var rv = MeshObj("Reverse", t, PaMesh.RoundBox(new Vector3(0.3f, 0.12f, 0.06f), 0.03f, 1), Lit("#DDE3EA", 0.6f), new Vector3(0, 0.62f, -2.06f), Quaternion.identity, false, false);`
`ParkAcademyGame.View.cs:435 / void BuildCarBody(Transform t, string tint, bool player)`	`_revL = rv.GetComponent<Renderer>(); _revOff = Lit("#DDE3EA", 0.6f); _revOn = Emit("#FFFFFF", 2.2f);`
`ParkAcademyGame.View.cs:437 / void BuildCarBody(Transform t, string tint, bool player)`	`MeshObj("LSign", t, PaMesh.RoundBox(new Vector3(0.7f, 0.42f, 0.08f), 0.05f, 1), Lit("#FFFFFF", 0.4f), new Vector3(0, 1.8f, -0.3f));`
`ParkAcademyGame.View.cs:438 / void BuildCarBody(Transform t, string tint, bool player)`	`MeshObj("L1", t, PaMesh.Box(new Vector3(0.09f, 0.28f, 0.1f)), Lit("#FF5A4E", 0.4f), new Vector3(-0.08f, 1.82f, -0.3f), Quaternion.identity, false, false);`
`ParkAcademyGame.View.cs:439 / void BuildCarBody(Transform t, string tint, bool player)`	`MeshObj("L2", t, PaMesh.Box(new Vector3(0.22f, 0.08f, 0.1f)), Lit("#FF5A4E", 0.4f), new Vector3(-0.01f, 1.71f, -0.3f), Quaternion.identity, false, false);`
`ParkAcademyGame.View.cs:447 / void BuildCar()`	`BuildCarBody(_carBody, "#7FE0C2", true);`

`ParkAcademyGame.View.cs:449 / void BuildCar()`	`MeshObj("Blob", _carT, PaMesh.Quad(2.3f, 4.6f), Fade("#24375A", 0.18f), Vector3.up * 0.015f, Quaternion.identity, false, false);`
`ParkAcademyGame.View.cs:450 / void BuildCar()`	`_shield = MeshObj("Shield", _carT, PaMesh.Sphere(1f, 18, 12), Glow("#7FF5E8", 0.22f), new Vector3(0, 0.9f, 0), Quaternion.identity, false, false);`
`ParkAcademyGame.View.cs:460 / void Palm(Transform parent, Vector3 pos, float s, int seed)`	`var trunk = Lit("#B88A5A", 0.2f);`
`ParkAcademyGame.View.cs:468 / void Palm(Transform parent, Vector3 pos, float s, int seed)`	`var leaf = Lit("#4DBF7A", 0.3f);`
`ParkAcademyGame.View.cs:475 / void Palm(Transform parent, Vector3 pos, float s, int seed)`	`MeshObj("Coco", p, PaMesh.Sphere(0.16f, 8, 6), Lit("#7A5530", 0.3f), at + Vector3.down * 0.15f);`
`ParkAcademyGame.View.cs:481 / void Umbrella(Vector3 pos, string a, string b, int seed)`	`MeshObj("Pole", u, PaMesh.Cylinder(0.05f, 2.4f, 8), Lit("#FFFFFF", 0.5f), new Vector3(0, 1.2f, 0));`
`ParkAcademyGame.View.cs:499 / void BuildDecor(int th)`	`string[] ca = { "#FF7A6B", "#2EC4B6", "#FFC94A", "#9D8CFF" };`
`ParkAcademyGame.View.cs:503 / void BuildDecor(int th)`	`Umbrella(W(x, hh + 13 + (float)r.NextDouble() * 6), ca[i % 4], "#FFFFFF", i);`
`ParkAcademyGame.View.cs:506 / void BuildDecor(int th)`	`var rail = Lit("#FFFFFF", 0.5f);`
`ParkAcademyGame.View.cs:511 / void BuildDecor(int th)`	`MeshObj("Hut", k, PaMesh.RoundBox(new Vector3(3f, 2.4f, 2.4f), 0.2f, 2), Lit("#FFF6E5", 0.4f), new Vector3(0, 1.2f, 0));`
`ParkAcademyGame.View.cs:512 / void BuildDecor(int th)`	`MeshObj("Awning", k, PaMesh.RoundBox(new Vector3(3.4f, 0.25f, 1.2f), 0.1f, 2), Lit("#FF7A6B", 0.4f), new Vector3(0, 2.3f, -1.5f), Quaternion.Euler(-15, 0, 0));`
`ParkAcademyGame.View.cs:513 / void BuildDecor(int th)`	`MeshObj("Cone", k, PaMesh.Cone(0.45f, 1.2f, 12), Lit("#E8B270", 0.3f), new Vector3(0, 3.0f, 0), Quaternion.Euler(180, 0, 0));`
`ParkAcademyGame.View.cs:514 / void BuildDecor(int th)`	`MeshObj("Scoop", k, PaMesh.Sphere(0.55f, 12, 8), Lit("#FFE3EC", 0.4f), new Vector3(0, 3.7f, 0));`

ParkAcademyGame.View.cs:516 / void BuildDecor(int th)	for (int i = 0; i < 5; i++) MeshObj("Ring", lh, PaMesh.Cylinder(1.6f - i * 0.15f, 2.4f, 16, 0.94f), Lit(i % 2 == 0 ? "#FFFFFF" : "#FF5A4E", 0.4f), new Vector3(0, 1.2f + i * 2.4f, 0));
ParkAcademyGame.View.cs:517 / void BuildDecor(int th)	MeshObj("Lamp", lh, PaMesh.Cylinder(0.9f, 1.4f, 12), Emit("#FFF2B0", 1.8f), new Vector3(0, 12.7f, 0));
ParkAcademyGame.View.cs:518 / void BuildDecor(int th)	MeshObj("Cap", lh, PaMesh.Cone(1.2f, 1.2f, 12), Lit("#24375A", 0.5f), new Vector3(0, 14f, 0));
ParkAcademyGame.View.cs:705 / void BuildStateMaterials()	_mLampOn = new[] { Emit("#FF4D4D", 3f), Emit("#FFC94A", 3f), Emit("#4DE08A", 3f) };
ParkAcademyGame.View.cs:706 / void BuildStateMaterials()	_mLampOff = Lit("#333C48", 0.6f);
ParkAcademyGame.View.cs:707 / void BuildStateMaterials()	_mBayHold = Emit("#FFE36A", 1.6f);
ParkAcademyGame.View.cs:708 / void BuildStateMaterials()	_mPlain = Lit("#FFFFFF", 0.3f);
ParkAcademyGame.View.cs:709 / void BuildStateMaterials()	_mStopDone = Lit("#C9F2D8", 0.3f);
ParkAcademyGame.View.cs:710 / void BuildStateMaterials()	_cBay = HzUI.Hex("#B8FFF2"); _cBooth = HzUI.Hex("#FFE08A"); _cWash = HzUI.Hex("#9FDBFF");
ParkAcademyGame.View.cs:711 / void BuildStateMaterials()	_mBayPulse = new Material(Emit("#B8FFF2", 1.5f));
ParkAcademyGame.View.cs:712 / void BuildStateMaterials()	_mStopPulse = new[] { new Material(Emit("#FFE08A", 1.5f)), new Material(Emit("#9FDBFF", 1.5f)) };
ParkAcademyGame.View.cs:726 / void ShowGuideDots()	var dot = Emit("#FFC23A", 1.1f);
ParkAcademyGame.View.cs:755 / void BuildFxPool()	_dustM = Fade("#F4E6D0", 0.55f); _splashM = Fade("#BDE9FF", 0.7f); _bubbleM = Glass("#E8FBFF", 0.55f); _sparkM = Glow("#FFE27A", 0.9f, 1.4f);
ParkAcademyGame.View.cs:826 / void OnStopDoneView(int i)	Toast(s.kind == (int)StopKind.Wash ? "Squeaky clean! Gate open." : "Ticket OK! Barrier up.", HzUI.Hex("#E3FFF6"), 1.2f, 0.7f);

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/ParkAcademyGame.UI.cs`, find the displayed literal `"Settings"`, and replace its displayed value with `"Options"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. Exact helper names/unlocks/charges: Scripts/ParkAcademyGame.cs; help and effect descriptions: Scripts/ParkAcademyGame.cs. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`ParkAcademyGame.cs:25 / class/static declarations`	`static readonly string[] BoostName = { "Guide Line", "Bumper", "Slow Tide", "Autopilot" };`
`ParkAcademyGame.cs:86 / IEnumerator BuildGenerated()`	`if (!toast && Time.realtimeSinceStartup - t0 > 0.25f) { toast = true; Toast("Painting a new lot...", null, 2.2f, 0.62f); }`
`ParkAcademyGame.cs:236 / void OnSimEvents(byte code)`	`if (_s.status == 0) Toast(StrikeText(), HzUI.Hex("#FFD1C9"), 1.1f, 0.7f);`
`ParkAcademyGame.cs:359 / void OnBooster(int b)`	`if (!Unlocked(b)) { HzAudio.Play(Sfx.Wrong, 0.5f); Toast(BoostName[b] + " unlocks at level " + BoostUnlock[b], null, 1.2f, 0.62f); return; }`
`ParkAcademyGame.cs:361 / void OnBooster(int b)`	`if (BoostActive(b)) { Toast(BoostName[b] + " is already on", null, 1.0f, 0.62f); return; }`
`ParkAcademyGame.cs:362 / void OnBooster(int b)`	`if (b == BSlow && !_def.Dynamic) { HzAudio.Play(Sfx.Wrong, 0.4f); Toast("No vans or gulls on this lot", null, 1.2f, 0.62f); return; }`
`ParkAcademyGame.cs:363 / void OnBooster(int b)`	`if (string.IsNullOrEmpty(_def.plan) && (b == BGuide || b == BAuto)) { Toast("Coach is still checking this lot", null, 1.2f, 0.62f); return; }`

ParkAcademyGame.cs:367 / void OnBooster(int b)	Popup("More " + BoostName[b] + "?", "Watch a short ad for 1 free " + BoostName[b] + ".", HzUI.Hex(BoostColor[b]),
ParkAcademyGame.cs:393 / void StartAutopilot()	Toast("Coach Pelly is driving!", HzUI.Hex("#FFE3D9"), 1.6f, 0.62f);
ParkAcademyGame.UI.cs:65 / void BuildHud()	_levelText = HzUI.Label("Lvl", lp.transform, "Lesson " + Level, 54, Color.white);
ParkAcademyGame.UI.cs:68 / void BuildHud()	_levelSub = HzUI.Label("Name", lp.transform, _def.name, 30, HzUI.Hex("#E3FFFB"), TextAnchor.MiddleCenter, false);
ParkAcademyGame.UI.cs:76 / void BuildHud()	var cam = HzIcons.IconButton(_hud, _camMode == 0 ? HzIcons.Eye : HzIcons.Grid, HzUI.Hex("#5B9DF0"), ToggleCamera, bs);
ParkAcademyGame.UI.cs:101 / void BuildHud()	_goalText = HzUI.Label("Goal", _strikePanel, "", 36, Ink, TextAnchor.MiddleLeft, true);
ParkAcademyGame.UI.cs:121 / void BuildHud()	var kh = HzUI.Label("Keys", _hud, "W A S D / arrows to drive - Space brake - C camera - R restart", 26, new Color(0.14f, 0.21f, 0.35f, 0.85f), TextAnchor.MiddleCenter, false);
ParkAcademyGame.UI.cs:140 / void BuildHud()	_tipText = HzUI.Label("T", _tipPanel, "", portrait ? 40 : 38, Ink, TextAnchor.MiddleCenter, true);
ParkAcademyGame.UI.cs:151 / void BuildHud()	var pIcon = HzUI.Label("P", _ring, "P", 70, Color.white); pIcon.rectTransform.Stretch(); pIcon.AddOutline(new Color(0.1f, 0.2f, 0.3f, 0.6f), 3);
ParkAcademyGame.UI.cs:192 / void BuildPads(bool portrait)	var l = HzUI.Label("Label", img.transform, label, 34, Color.white); l.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(size.x, 50), new Vector2(0, 34));
ParkAcademyGame.UI.cs:205 / void BuildPads(bool portrait)	v.Btn = HzUI.Button("Booster" + b, parent, null, Color.white, Color.white, () => OnBooster(id), 40, 75);
ParkAcademyGame.UI.cs:227 / void BuildPads(bool portrait)	v.BadgeText = HzUI.Label("N", bd.transform, "", 34, Color.white); v.BadgeText.rectTransform.Stretch();
ParkAcademyGame.UI.cs:231 / void BuildPads(bool portrait)	var ot = HzUI.Label("T", on.transform, "ON", 28, Color.white); ot.rectTransform.Stretch();

`ParkAcademyGame.UI.cs:233 / void BuildPads(bool portrait)`	`v.Caption = HzUI.Label("Cap", v.Root, "", 30, Color.white);`
`ParkAcademyGame.UI.cs:253 / void RefreshBoosters()`	`v.BadgeText.text = c > 0 ? c.ToString() : "+";`
`ParkAcademyGame.UI.cs:255 / void RefreshBoosters()`	`v.Caption.text = un ? BoostName[b] : "Lv " + BoostUnlock[b];`
`ParkAcademyGame.UI.cs:317 / void LevelBanner()`	`var t = HzUI.Label("L", bg.transform, "Lesson " + Level, 92, Color.white); t.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(800, 110), new Vector2(0, -16), new Vector2(0.5f, 1));`
`ParkAcademyGame.UI.cs:320 / void LevelBanner()`	`var n = HzUI.Label("N", bg.transform, sub, 44, HzUI.Hex("#BDF5EC")); n.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(800, 70), new Vector2(0, 18), new Vector2(0.5f, 0));`
`ParkAcademyGame.UI.cs:400 / void CompleteGuide(int b, bool toast = true)`	`if (toast) Toast("Nice! " + BoostName[b] + " is on", HzUI.Hex("#E3FFF6"), 1.3f, 0.62f);`
`ParkAcademyGame.UI.cs:477 / HzPopup AcademyPopup(string title, string message, Color accent, Sprite art, params (string label, Color color, Action action)[] buttons)`	`var t = HzUI.Label("Title", ribbon.transform, title, 66, Color.white); t.rectTransform.Stretch(6);`
`ParkAcademyGame.UI.cs:482 / HzPopup AcademyPopup(string title, string message, Color accent, Sprite art, params (string label, Color color, Action action)[] buttons)`	`p.Message = HzUI.Label("Msg", card.transform, message, 40, Ink, TextAnchor.MiddleCenter, false);`
`ParkAcademyGame.UI.cs:514 / void UnlockPopup(int b, Action done)`	`AcademyPopup("New: " + BoostName[b] + "!", BoostDesc[b] + "\nYou get " + BoostStart[b] + " free!", HzUI.Hex(BoostColor[b]), Art(BoostArt[b]),`
`ParkAcademyGame.UI.cs:576 / void MechanicIntro(string m, Action done)`	`var p = AcademyPopup("New: " + MechanicTitle(m), MechanicText(m), Coral, null, ("Got it!", Teal, done));`
`ParkAcademyGame.UI.cs:586 / void WinPopup(int lv, int stars, string msg, Action next)`	`var p = AcademyPopup("Parked!", msg + "\nLesson " + lv + " complete", Teal, Art(stars == 3 ? "win_badge" : "car_icon"),`
`ParkAcademyGame.UI.cs:606 / void FailPopup(string title, string msg, (string, Color, Action)[] buttons)`	`AcademyPopup(title, msg, Coral, Art("lose_badge"), buttons);`

`ParkAcademyGame.UI.cs:611 / void ShowSettingsPopup()`	`var p = AcademyPopup("Settings", "", Ink, null,`
`ParkAcademyGame.UI.cs:612 / void ShowSettingsPopup()`	`(SoundLabel(), HzUI.Hex("#5B9DF0"), null),`
`ParkAcademyGame.UI.cs:613 / void ShowSettingsPopup()`	`(MusicLabel(), HzUI.Hex("#9D8CFF"), null),`
`ParkAcademyGame.UI.cs:629 / void ShowHowTo()`	`var p = AcademyPopup("How to Play", text, HzUI.Hex("#FFB938"), Art("coach_pelly"), ("Let's drive!", Teal, null));`
`ParkAcademyGame.UI.cs:633 / static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`	`static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`
`ParkAcademyGame.UI.cs:634 / static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`
`ParkAcademyGame.View.cs:826 / void OnStopDoneView(int i)`	`Toast(s.kind == (int)StopKind.Wash ? "Squeaky clean! Gate open." : "Ticket OK! Barrier up.", HzUI.Hex("#E3FFF6"), 1.2f, 0.7f);`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/parkacademy through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Effects are under Resources/parkacademy/Sfx. Preserve each actual load key and importer settings when replacing sounds. Supplied music: Resources/parkacademy/Music/seaside_loop.ogg. Preserve that load key and verify loop/compression and the independent Music toggle. Kit UI uses supplied fonts; preserve actual licence notices and verify new glyphs.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed arena, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.parkacademy`, version, scene orientation and signing. The build command writes `Builds/Android/parkacademy.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/parkacademy/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.